Adroit Technologies Protocol Driver

Name	MQTT Client Protocol Driver
DLL	D_MQTT_CL

COPYRIGHT

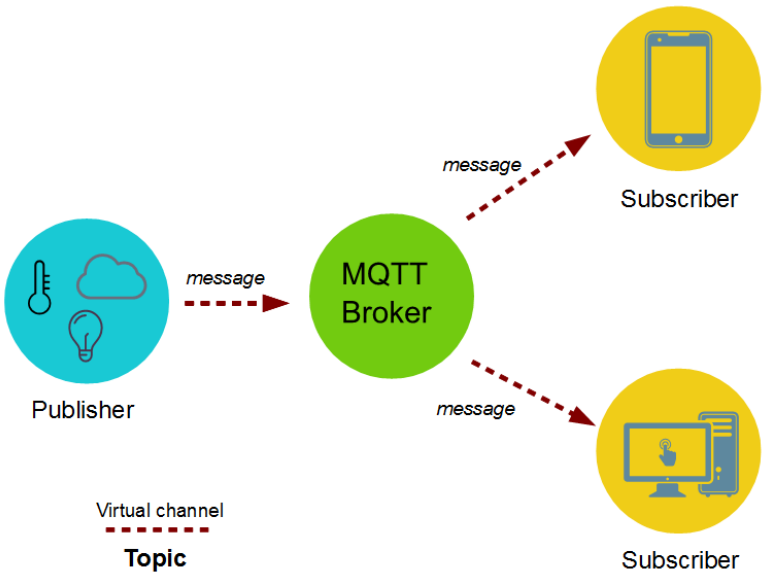
Advanced Worx 112 (Pty) Ltd
20 Waterford Office Park
189 Witkoppen Road (c/o Waterford Drive)
Fourways
South Africa

https://adroittech.co.za/

Contents

1. Overview

Message Queue Telemetry Transport (MQTT) is a lightweight, report by exception, ISO-standard pub-sub messaging protocol specifically designed for the Internet of Things. The MQTT network consists of publishers, subscribers and a broker. The publishers will publish or send information to the broker, the broker will then relay those messages to the subscribers. The broker is the "middleman" in the network and could be situated in the cloud. A publisher/subscriber are the clients on the network and are linked together by topics.



Each Adroit agent can be a publisher and/or subscriber, and acts as a client on the network. If Adroit is a publisher, then it will write values out to other subscribers (when the tag is scanned using **Output enabled**). When Adroit is a subscriber, then it waits for data on a specific topic from any publisher to that topic.

There are three parts of an MQTT transmission:

1. **Message**: This is the data that is passed through the broker to all clients subscribed to a specific topic on the MQTT network.
2. **Topic**: Some brokers support topic strings with a maximum of 128 levels and a maximum length of 250 bytes, so an MQTT client connection that exceeds those limits will be closed.
3. **QoS**: Quality of Service, ensures delivery to connected clients:
   a. At most once - The published message is sent once, and if it does not arrive it is lost.
   b. At least once - Guarantees that the published message arrives, but there may be duplicates.
   c. Exactly once - Guarantees that the publish message arrives and that there are no duplicates.

1.1. Supported Hardware

This driver has been tested on the following hardware:

None applicable.

The driver has been tested is known to work with the following brokers:

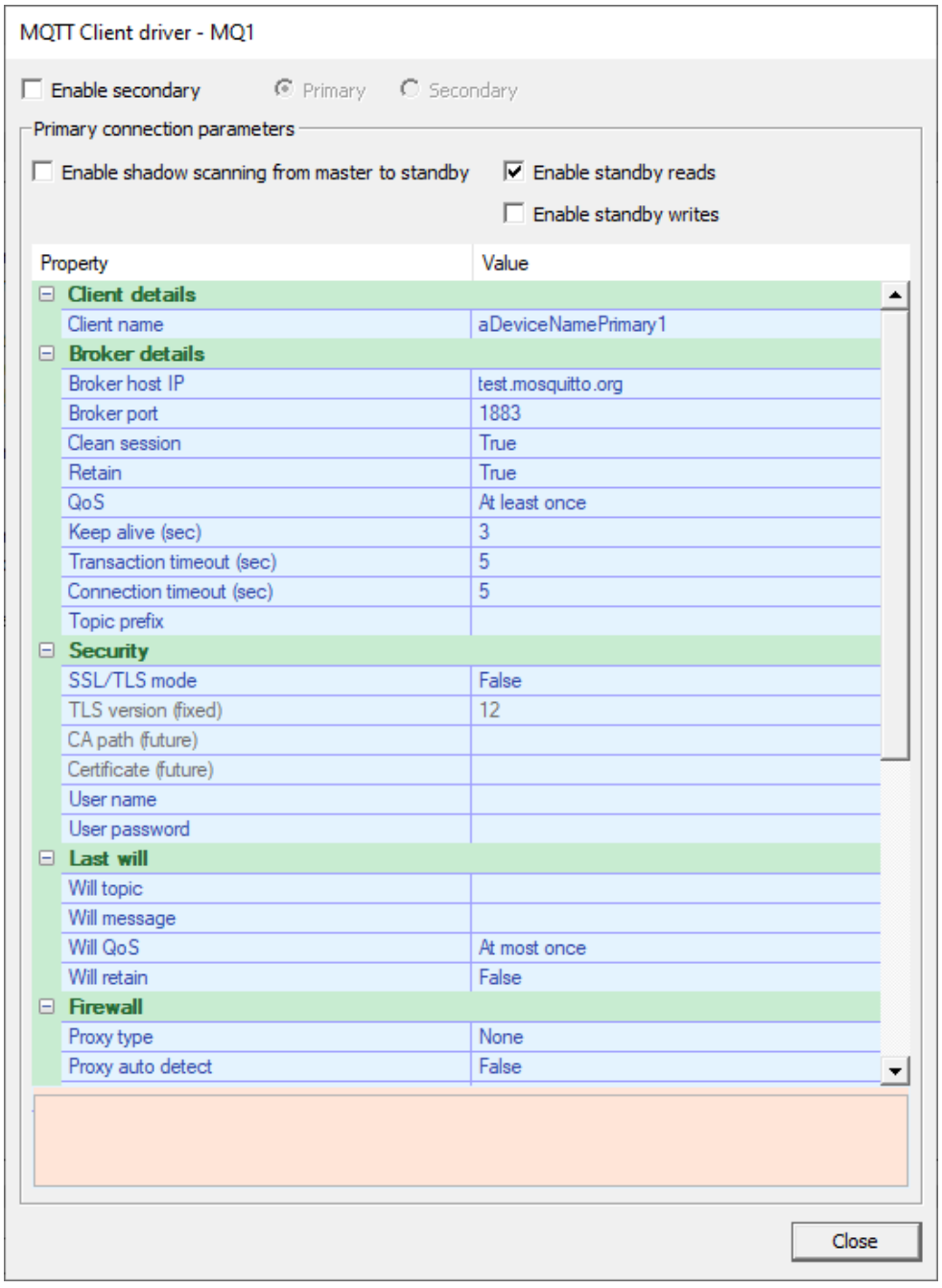
- Mosquitto
- HiveMQ

2. Wiring

Not applicable.

3. Device Configuration

3.1. Device Configuration



Enable secondary – Turn this option to have the driver maintain 2 channels of communication, the radio buttons **Primary** and **Secondary** can then be used to toggle between configurations.

3.1.1. Driver Details

Enable Secondary – (Default = OFF) Allows enabling of redundant MQTT Broker connections such that in case of a failure a switchover will occur.

Primary – Allows configuration setting of the Primary broker connection.

Secondary – Allows configuration setting of the Secondary broker connection.

3.1.2. Device Details

Enable shadow scanning from master to standby – (Default = OFF) If enabled will stop any communication to the MQTT broker when the Agent server is in standby mode. By default, we assume parallel scanning.

Enable standby reads – (Default = ON) Allows reading from the MQTT broker when agent server is in standby mode.

Enable standby writes – (Default = OFF) Allows writing to the MQTT broker when agent server is in standby mode.

Broker Host IP – The static IP Address or HOST name of the MQTT broker. Domain names are resolved to IP addresses. To connect over Websockets specify a hostname beginning with ws:// or wss://.
For example: ws://test.mosquitto.org or test.mosquitto.org

Port – A valid port number is required for the connection to take place.

Client Name – The client identifier is a (1- 23 UTF-8 encoded) unique name to identify each MQTT client to the MQTT broker. It should be unique per client for a given broker, and each Adroit server device needs its own unique client ID. Click the button on the right-hand side to generate a random string for this field if required.

Username – A username could be required by the MQTT broker. This allows authentication and authorization of clients to a broker. This must be a UTF-8 encoded string.

Password – A password could be required by the MQTT broker. This allows authentication and authorization of clients to a broker.

QoS – (Default = 2) Quality of Service level of each client. The level (0, 1 or 2) determines the guarantee of a message being delivered.

Clean Session – (Default = True) This will tell the MQTT broker to establish a persistent session or not. If True, the broker will discard any data previously associated with the current client ID once successfully connected. Otherwise, when disconnected the broker and client will store the session, all subscriptions for the client and all missed messages, when QoS 1 or 2.

Retain Flag – (Default = False) When publishing a non-empty message with the Retain flag set and a non-zero QoS level, will cause the server to store it (replacing any previously retained message in the process). When the message is stored, any new clients that subscribe to that topic will receive the message.

Auto Reconnect – (Default = False) The MQTT driver will automatically reconnect to the MQTT broker if connection is lost.

KeepAliveTime – The KeepAliveTime is a time interval that the client will continuously send control packets/PING requests to the MQTT broker to show that the client is still alive. If the broker does not receive a control packet/PING request within 1.5 times the KeepAliveTime, the broker MUST disconnect the network connection to the client as if the network had failed.

Timeout – (Default = 60) If the Timeout is set to 0, all operations return immediately, potentially failing with an error if not completed immediately. If set to a positive value, then the driver will wait for the operation to complete before returning control.

Retries – Number of retries before trying to switch between primary/secondary. Each Retry is 1 second.

Will Topic – The topic to publish will payload. Can have a hierarchal structure with forward slashes as delimiters.

Will Message – The message to publish when the Agent Server disconnects ungracefully.

Will QoS Level – (Default = 0) Publish payload with QoS set.

Will Retain Flag – (Default = False) This flag determines if the will message will be saved by the broker for the specific topic. New clients that subscribe to that topic will receive the last retained message on that topic after subscribing.

4. Supported Addresses

The scan address in Adroit is the topic in the MQTT protocol with a format prefix. A MQTT topic is a simple string with a hierarchical structure, with forward slashes as delimiters.

For example:

Prefix:City/house/bedroom/lamp/on

> **PLEASE NOTE**
>
> Currently the Adroit MQTT driver does not support wildcards, so please avoid using "+" and "#" characters in the scan addresses.

Prefixes :

Data type	Prefix	Example
String	S	S:MYSTRING
Long	L	L:MYSIGNED32
Double	D	D:MYUNSIGNED32
Integer	I	I:MYSIGNED32
Word	W	W:MYSIGNED32
Long	F	F:MYSIGNED32

> **INFORMATION**
>
> Prefixes attempt to force the driver to pre-convert the I/O to various types and must be included.

4.1. Example

Making use of the online MQTT Broker supplied by Mosquito you can test the MQTT functionality. Settings is as follows:

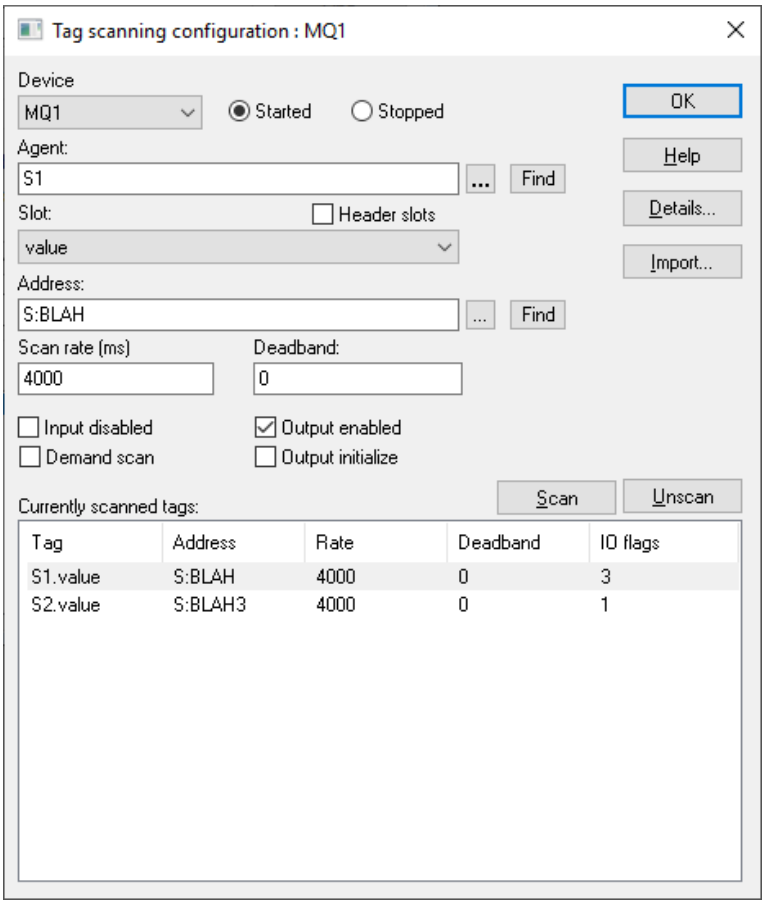
HOST: test.mosquitto.org
PORT: 1883

The scan address can be anything, as it relates to the Topic name. On this broker, scanning to any topic will register the topic and allow values to be transferred through the channel.

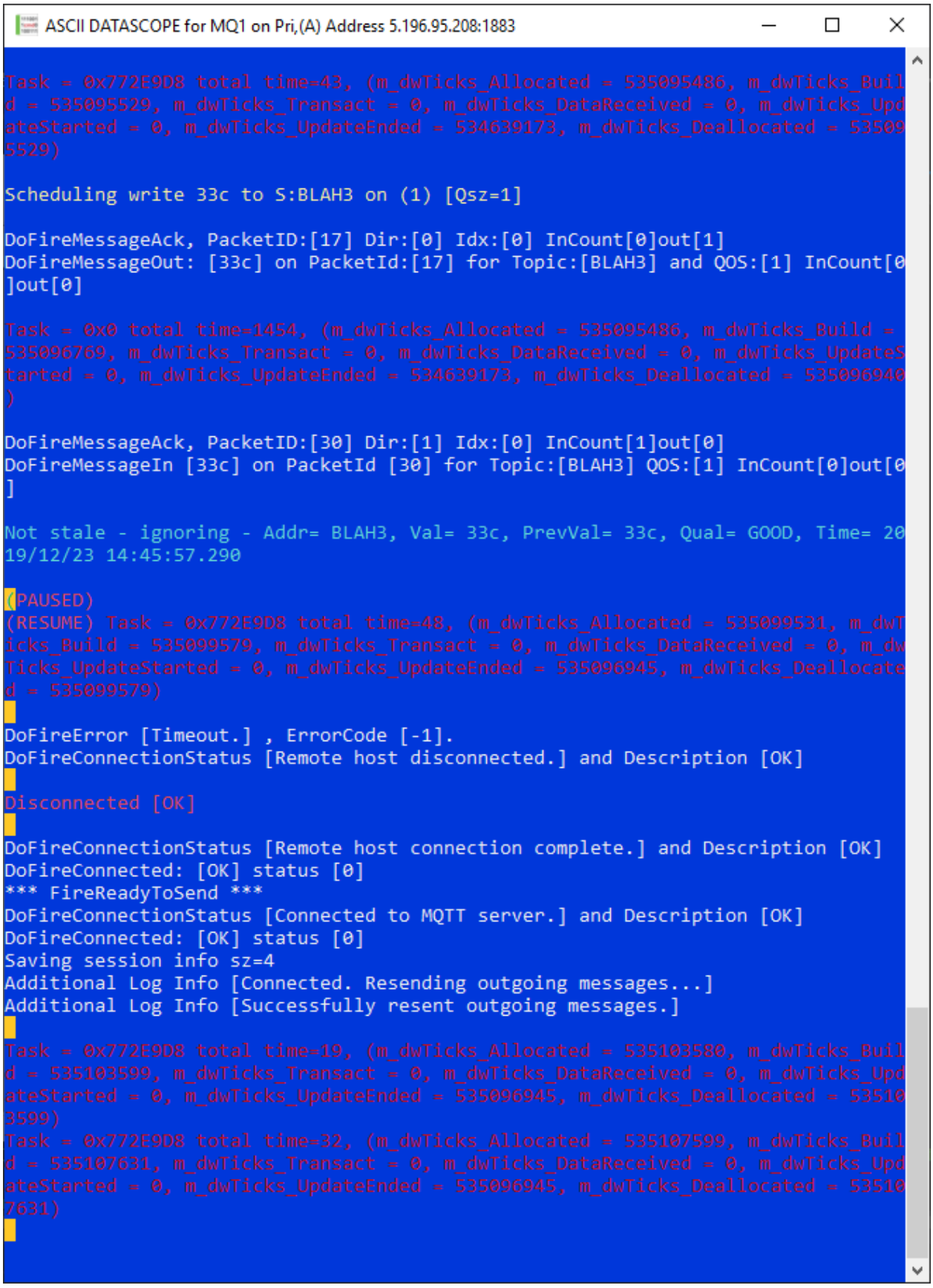
Scanning a string agent to the broker will now have the following result in the scanniong dialog.

Other supported ports for various Mosquitto.org test connections:

- 1883 : MQTT, unencrypted
- 8883 : MQTT, encrypted
- 8884 : MQTT, encrypted, client certificate required
- 8080 : MQTT over WebSockets, unencrypted
- 8081 : MQTT over WebSockets, encrypted



5. Protocol Definition and Considerations



5.1. Additional MQTT Information

Retained message: A retained flag tells the MQTT broker to store the last message of each topic, in case a new client subscribes to the broker and automatically deliver the message to that new client. Retained flag off means if a client is offline during a publish, it will never get that message.

Persistent Sessions: This makes MQTT designed for intermittent connectivity. Sessions may last weeks, months or years and it does this using a keep alive message and a timeout. QoS 1 & 2 messages are queued for clients which may be offline but have not timed out.

Last Will & Testament messages: If enabled, a device will register a message to the broker so that if a session is lost, send this message to a specific topic. This can be used to see which devices are online/offline at any given time.

Quality of Service levels explained:

- **QoS 0:** At most once, "Fire and forget". Sends a message and does not wait for a reply. Sender never retries to send the message and receiver might not get message.
- **QoS 1:** At least once, "guaranteed delivery", continuously sends a message until there is a reply from the broker. But there is a chance of a duplicate message on receiver side while waiting for confirmation.
- **QoS 2:** Exactly once, "2 phase commit guaranteed delivery". This is used when packet losses or duplication of messages is unacceptable. Comes at additional network traffic, most data heavy service so not ideal for GSM or GPRS.

MQTT Ports:

1. TCP: 1883
2. TCP + SSL: 8883
3. Websocket: 9001
4. Websocket + SSL: 9883

MQTT Advantages:

- Very lightweight protocol, devices do not need a lot of resources such as RAM. High power efficiency.
- The protocol supports for messages up to 256 MB is size.
- Publishers and Subscribers can communicate to each other without knowing ports, IP addresses etc.
- Devices don't have to be actively connected at the same time. Therefore suitable for unstable networks.
- Sending and receiving messages happens at each devices own speed, synchronization works seamlessly.

6. General Notes and Troubleshooting

6.1. Contact Details

Email: support@adroit.co.za and drivers@adroit.co.za

Phone: +27 11 658-8100

Web: www.adroit.co.za

6.2. Adroit Technologies Knowledge Base

For additional information, please refer to our Knowledgebase website:

Latest Drivers topics - Features, discussions, tips, tricks, questions, problems and feedback (adroittechnologiesautomation.com)

7. Revisions

7.1. Document Revision

Rev.	Date	Author	Comment	Approval
1.0	14-Dec-2022	J. Duncan	New Format	D. Jordaan
1.1	09-Jan-2023	K. Robinson	Altered some text formatting, added more detailed descriptions.	D. Jordaan
2.0				
3.0				
4.0				
5.0				
6.0				

7.2. DLL Revision

Rev.	Date	Changes
1	30/07/2019	First release of the driver.
24	01/12/2022	Shadow mode partner (with offline redundant plc) in standby did not mark affected tags bad when becoming master. For Shadow and Parallel , when switching cluster : unchanging tags were not updated. Updates for first update now working as expected.